

Interactive 2D Mobile App Development To-Do List

For COC1–COC4 Mission Design, Gamified Interaction, Evidence Capture, and Final QA

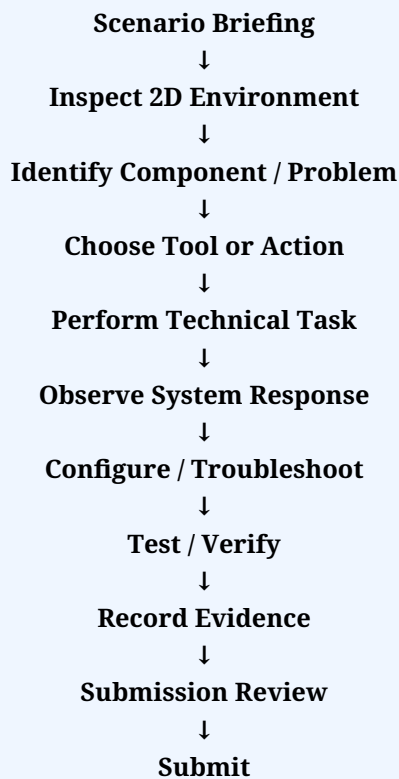
Core design standard: Every mission should make the learner perform a simulated CSS task, observe a result, make a decision, and produce evidence — not simply answer a question.

Prepared as a practical implementation checklist for the ByteQuest Flutter mobile application.

Phase 1 — Establish One Quality Standard for All 20 Missions

Apply the following baseline to every mission across COC1–COC4, Mission 1–5:

- ☐ Every mission has a clear scenario or technical problem.
- ☐ Every mission has at least 3–6 meaningful interaction phases.
- ☐ Every mission uses at least 2–4 different interaction types.
- ☐ Drag-and-drop is never the only mechanic.
- ☐ Every mission includes at least one technical decision.
- ☐ Every mission includes observation, testing, or verification where appropriate.
- ☐ Every meaningful action produces structured evidence.
- ☐ Incorrect actions provide useful feedback without revealing the answer.
- ☐ Assessment mode does not expose correct targets.
- ☐ Practice mode may provide additional hints.
- ☐ Mission state survives pause/resume.
- ☐ No duplicate evidence is created after reconnect/resume.
- ☐ Every gesture-heavy interaction has an accessible alternative.
- ☐ All missions work on different Android screen sizes.
- ☐ Every mission has a polished submission-review stage.



Phase 2 — Build the Reusable Interaction System First

Avoid hard-coding 20 unrelated mission screens. Build reusable Flutter components and compose missions from them.

Core 2D Scene

- ☐ Responsive SimulationScene
- ☐ Zoom
- ☐ Pan
- ☐ Fit scene
- ☐ Reset camera
- ☐ Optional fullscreen
- ☐ Landscape support
- ☐ Responsive object positioning
- ☐ Neutral interactive hotspots
- ☐ Selected-object state
- ☐ Completed-object state
- ☐ Error feedback
- ☐ Scene-state persistence

Reusable Interaction Components

- ☐ Tap / Inspect
- ☐ Multi Select
- ☐ Tool Selection
- ☐ Connection
- ☐ Configuration
- ☐ Sequencing
- ☐ Matching
- ☐ Controlled Placement
- ☐ Troubleshooting
- ☐ Testing
- ☐ Observation
- ☐ Scenario Decision
- ☐ Result Interpretation
- ☐ Evidence Review

Tool System

- ☐ Select a tool from a reusable tool tray.
- ☐ Apply the tool only to compatible objects.
- ☐ Reject invalid usage with concise technical feedback.
- ☐ Show a simulated result after the action.
- ☐ Save selected tool/action as structured evidence.

Phase 3 — COC1: Install and Configure Computer Systems

COC1 should feel focused on components, assembly, installation, device setup, configuration, and verification.

Mission	Recommended Interactive Experience
M1	Hardware inspection + component identification
M2	Component installation / assembly

M3	System installation / configuration
M4	Device / peripheral configuration
M5	Verification + troubleshooting

COC1 Mission 1 — Inspection and Hardware Identification

- ☐ Display a complete 2D computer/workbench scene.
- ☐ Allow tap-to-inspect components.
- ☐ Use multi-select for relevant components.
- ☐ Allow inspection of ports/components.
- ☐ Add tool identification.
- ☐ Add component classification.
- ☐ Require identification of abnormalities.
- ☐ Include an observation-record step.
- ☐ Finish with verification.

COC1 Mission 2 — Controlled Assembly Simulation

- ☐ Select component.
- ☐ Select destination.
- ☐ Validate compatibility.
- ☐ Validate sequence.
- ☐ Validate orientation where relevant.
- ☐ Install component.
- ☐ Connect required simulated connections.
- ☐ Show installed state.
- ☐ Verify final assembly.

COC1 Mission 3 — Installation / Configuration Workflow

- ☐ Boot/setup scenario.
- ☐ Select configuration actions.
- ☐ Sequence installation steps.
- ☐ Use a simulated configuration panel.
- ☐ Support driver/component selection where appropriate.
- ☐ Detect incorrect configuration.
- ☐ Simulate restart/verification.
- ☐ Require result interpretation.

COC1 Mission 4 — Peripheral / Device Configuration

- ☐ Identify peripheral.
- ☐ Select correct connection.
- ☐ Connect device.
- ☐ Configure appropriate setting.
- ☐ Inspect device status.

- ☐ Run simulated test.
- ☐ Interpret result.
- ☐ Fix incorrect configuration.

COC1 Mission 5 — Integration / Troubleshooting Scenario

- ☐ Inspect symptoms.
- ☐ Choose diagnostic action.
- ☐ Select tool.
- ☐ Perform simulated test.
- ☐ Interpret result.
- ☐ Identify likely problem.
- ☐ Perform corrective action.
- ☐ Run final verification.

Phase 4 — COC2: Set Up Computer Networks

COC2 should feel visibly different from COC1 by emphasizing connections, topology, configuration, testing, and troubleshooting.

COC2 Mission 1 — Cabling / Network Preparation

- ☐ Identify network devices.
- ☐ Select appropriate cable/connection.
- ☐ Select required tool.
- ☐ Sequence preparation steps.
- ☐ Use controlled source → destination connection.
- ☐ Run simulated cable/network test.
- ☐ Interpret test output.

COC2 Mission 2 — Preserve the Golden / Reference Mission

- ☐ Preserve its validated authoritative evaluation rules.
- ☐ Keep topology interaction where applicable.
- ☐ Use device selection.
- ☐ Use source/destination connection.
- ☐ Use configuration activities.
- ☐ Include connectivity testing.
- ☐ Include result interpretation.
- ☐ Improve visuals only when safe.

COC2 Mission 3 — Network Topology Construction

- ☐ Inspect requirements.
- ☐ Select devices.
- ☐ Choose connections.
- ☐ Connect nodes.
- ☐ Build topology.
- ☐ Verify link status.
- ☐ Fix invalid connections.
- ☐ Draw clear connection paths.

COC2 Mission 4 — Network Configuration

- ☐ Select device.
- ☐ Enter/select mission-appropriate network settings.
- ☐ Apply changes.
- ☐ Run connectivity test.
- ☐ Interpret result.
- ☐ Correct configuration errors.

COC2 Mission 5 — Network Troubleshooting

- ☐ Inspect topology.
- ☐ Test connection.
- ☐ Inspect configuration.
- ☐ Identify possible fault.
- ☐ Choose diagnostic action.
- ☐ Apply fix.
- ☐ Retest.
- ☐ Use branching technical decisions instead of one multiple-choice question.

Phase 5 — COC3: Set Up Computer Servers

COC3 should feel like a server administration simulator rather than a PC assembly exercise.

COC3 Mission 1 — Server Preparation

- ☐ Inspect server environment.
- ☐ Identify requirements.
- ☐ Select role/configuration.
- ☐ Inspect network readiness.
- ☐ Sequence setup preparation.
- ☐ Validate setup.

COC3 Mission 2 — Server Installation

- ☐ Use simulated setup/configuration flow.
- ☐ Include role selection.
- ☐ Make choices affect later simulated state.
- ☐ Allow configuration before installation.
- ☐ Simulate restart.
- ☐ Verify installation.
- ☐ Generate meaningful issues for incorrect configuration.

COC3 Mission 3 — Users / Groups / Permissions

- ☐ Create/select account.
- ☐ Assign role/group.
- ☐ Assign appropriate permissions.
- ☐ Inspect effective access.
- ☐ Test access.
- ☐ Diagnose permission errors.

COC3 Mission 4 — Server / Network Services

- ☐ Inspect services.
- ☐ Configure service.
- ☐ Start/stop appropriate simulated service.
- ☐ Configure required values.
- ☐ Inspect status.
- ☐ Test client access.
- ☐ Interpret server response.

COC3 Mission 5 — Server Troubleshooting

- ☐ Inspect client.
- ☐ Inspect server.
- ☐ Inspect service state.
- ☐ Test connectivity.
- ☐ Inspect permissions/configuration.
- ☐ Select corrective action.
- ☐ Retest.
- ☐ Verify recovery.

Phase 6 — COC4: Maintain and Repair Computer Systems and Networks

COC4 should be the strongest diagnostic and troubleshooting COC in the mobile app.

COC4 Mission 1 — Inspection

- ☐ Inspect environment.
- ☐ Identify symptoms.
- ☐ Select components to inspect.
- ☐ Record observations.
- ☐ Choose diagnostic priority.
- ☐ Build preliminary diagnosis.

COC4 Mission 2 — Hardware Diagnosis

- ☐ Inspect symptom.
- ☐ Choose test/tool.
- ☐ Inspect component.
- ☐ Run simulated test.
- ☐ Interpret result.
- ☐ Identify fault.
- ☐ Select repair/replacement action.
- ☐ Verify.

COC4 Mission 3 — Software / Network Diagnosis

- ☐ Use branching troubleshooting.
- ☐ Reveal new information based on the learner's diagnostic action.
- ☐ Require interpretation of each result.
- ☐ Avoid revealing the root cause too early.

COC4 Mission 4 — Repair / Corrective Action

- ☐ Component selection.
- ☐ Tool selection.
- ☐ Controlled replacement.
- ☐ Reconfiguration.
- ☐ Repair sequence.
- ☐ Verification.
- ☐ Post-repair test.

COC4 Mission 5 — Final Maintenance Scenario

- ☐ Receive maintenance request.
- ☐ Inspect system.
- ☐ Identify issues.
- ☐ Prioritize faults.
- ☐ Choose tools.
- ☐ Perform maintenance.
- ☐ Repair/configure.
- ☐ Test system.
- ☐ Interpret results.
- ☐ Final verification.
- ☐ Submission report.

Phase 7 — Add Proper Technical Feedback

- ☐ Use concise technical feedback instead of generic “Wrong”.
- ☐ Explain what failed without revealing the answer.
- ☐ Use brief success feedback after a verified action.
- ☐ Keep feedback short enough not to interrupt flow.

Phase 8 — Add Scenario State Changes

- ☐ Update scene state when a cable is connected.
- ☐ Change device status after correct configuration.
- ☐ Show installed component state after completion.
- ☐ Reflect test status/result visually.
- ☐ Ensure scene state is driven by real mission state, not decoration only.

Phase 9 — Add Troubleshooting Branching

- ☐ Do not expose all diagnostic information at the start.
- ☐ Reveal new symptoms/results after learner actions.
- ☐ Make technical decisions influence what information becomes available.
- ☐ Keep branching evidence-producing and meaningful.

Phase 10 — Gamification Around the Simulation

- ☐ Use mission progress and checkpoint feedback.
- ☐ Use approved progress/achievement feedback only.

- ☐ Keep gamification separate from official competency evaluation.
- ☐ Never invent TESDA XP, fake passing percentages, fake leaderboards, or random streaks.

Phase 11 — Make Submission Review Meaningful

- ☐ Show completed mission steps.
- ☐ Show evidence count.
- ☐ Allow learner to return to mission where permitted.
- ☐ Require explicit confirmation before authoritative submission.
- ☐ Keep review readable and compact.

Phase 12 — Preserve Assessment Authority

- ☐ Flutter records interactions and evidence.
- ☐ Supabase/PostgreSQL remains authoritative.
- ☐ Keep automated evaluation on the trusted backend.
- ☐ Keep provisional result, Instructor review, finalization, release, and learner result clearly separated.

Phase 13 — Preserve Realtime Integration

- ☐ Learner submission should update Instructor review workflow.
- ☐ Instructor release should update learner Mobile result.
- ☐ Progress and analytics should stay synchronized.
- ☐ Avoid local-only mission state that bypasses Supabase.

Phase 14 — Accessibility

- ☐ Provide select-item → select-destination alternatives to dragging.
- ☐ Provide source → target alternatives to connection gestures.
- ☐ Ensure small scene objects have accessible target sizes or an object list.
- ☐ Add Flutter Semantics.
- ☐ Support large text and reduced motion.

Phase 15 — Premium Visual Design

- ☐ Use ByteQuest blue/navy consistently.
- ☐ Use compact professional typography.
- ☐ Give the 2D workspace most of the screen.
- ☐ Use subtle borders and restrained shadows.
- ☐ Use smooth 150–250 ms feedback transitions.
- ☐ Avoid giant cards, giant logos, excessive gradients, glassmorphism, glow, or childish game styling.

Phase 16 — Add Subtle State Animations

- ☐ Animate cable connection by drawing the line into position.
- ☐ Fade status indicators on when device state changes.
- ☐ Snap/fade installed components into their completed state.
- ☐ Use subtle progress animation while tests run.
- ☐ Keep mission-step completion transitions compact.

Phase 17 — Mission Pause / Resume

- ☐ Restore the same attempt after app restart.
- ☐ Restore the current mission phase.
- ☐ Restore accepted evidence.
- ☐ Restore configuration and connection state.
- ☐ Do not duplicate evidence on resume.
- ☐ Do not trigger premature evaluation.

Phase 18 — Mission Quality Scorecard

Score every mission after implementation. These targets should be used as the acceptance standard:

Category	Target
Scenario realism	$\geq 4/5$
Interaction variety	$\geq 4/5$
Technical relevance	$\geq 4/5$
Decision depth	$\geq 3/5$
2D scene quality	$\geq 4/5$
Evidence quality	$\geq 4/5$
Accessibility	$\geq 4/5$
Visual polish	$\geq 4/5$
Persistence	5/5
Assessment integrity	5/5

Final target: 20/20 functional · 20/20 acceptable or rich · 0 weak · 0 form-only · 0 drag-drop-only missions

Phase 19 — Select Showcase Missions for Capstone Defense

- ☐ COC1 — installation/configuration showcase
- ☐ COC2 — networking/topology showcase
- ☐ COC3 — server configuration showcase
- ☐ COC4 — troubleshooting/repair showcase

Choose four missions that clearly demonstrate different interaction types so the panel immediately sees that ByteQuest is a 2D simulation platform, not just an LMS.

Phase 20 — Final Testing and Acceptance

After implementation, run:

```
flutter test
flutter analyze --no-fatal-infos
flutter build apk --debug
```

Manually verify COC1 M1–M5, COC2 M1–M5, COC3 M1–M5, and COC4 M1–M5. For every mission, verify:

- ☐ Launch
- ☐ Scenario displays
- ☐ 2D scene responsive
- ☐ Interactions work
- ☐ Incorrect states work
- ☐ Evidence saves
- ☐ Pause works
- ☐ Resume works
- ☐ Submission review works
- ☐ Submit works
- ☐ Evaluation works
- ☐ Realtime works
- ☐ Instructor sees result
- ☐ Release works
- ☐ Learner sees result
- ☐ No crashes
- ☐ No overflow
- ☐ No dead buttons

Final Product Experience Target

ByteQuest should not feel like: Read → Drag Item → Answer → Next.
It should feel like: Inspect → Interact → Diagnose → Configure → Connect → Test → Interpret →
Troubleshoot → Verify → Submit Evidence.

This interaction loop is the standard that makes ByteQuest defensible as a scenario-based 2D gamified CSS NC II simulation platform.